Git e GitHub; i formati dei file

3INF · lezione del 28/09/2026

Versione 1.0 · 29/10/2026

Corso a cura del prof. Nicola Regge

Che cosa abbiamo fatto

1. **Quando:** 28/09/2026, 3 ore, Laboratorio.
2. **Argomenti:** Differenza tra Git e GitHub; i formati di file più noti.

La teoria e l'esercitazione

Dispensa facile

Git & GitHub

raccontati con parole semplici

Alla fine di questa dispensa saprai **cos'è Git**, **a cosa serve**, che differenza c'è con **GitHub**, come funziona il **versioning** e vedrai **tanti esempi** di come si usa davvero. Niente comandi da digitare: qui capiamo le idee. Classe 3 · Informatica.

Di cosa parliamo

1. Partiamo da un problema che conosci già **2.** Cos'è Git, in due parole **3.** Un po' di storia: chi l'ha inventato e perché **4.** Git e GitHub non sono la stessa cosa **5.** GitHub è un *servizio*: PaaS, la famiglia …aaS (IaaS/SaaS) e il Freemium **6.** Le parole di Git spiegate facili (repository, commit, branch, merge) **7.** Come si lavora davvero (la storia di due compagni) **8.** Il versioning: i numeri 1.2.3 e come si fa **9.** Tanti casi d'uso con i grafi **10.** Riepilogo e glossario

1. Partiamo da un problema che conosci già

Immagina di dover consegnare una ricerca. La scrivi al computer e, per non perderla, ogni tanto la salvi con un nome nuovo. Dopo qualche giorno sul tuo desktop c'è questa scena:

ricerca.doc · ricerca_v2.doc · ricerca_BUONA.doc · ricerca_DEFINITIVA.doc · ricerca_DEFINITIVA_vera_QUESTA.doc

Ti suona familiare, vero? È un pasticcio. E nascono tre problemi:

1. Non sai più **qual è l'ultima** davvero buona, né **cosa è cambiato** da una all'altra.
2. Se cancelli una frase importante e te ne accorgi dopo, **non torni indietro**.
3. Se tu e un compagno lavorate sullo stesso file e ve lo passate, uno finisce per **cancellare** il lavoro dell'altro senza volerlo.

Ecco: **Git nasce per risolvere esattamente questo**. E lo fa in modo elegante.

2. Cos’è Git, in due parole

Git è come una **macchina del tempo** per i tuoi file. Tu lavori normalmente, e ogni volta che vuoi dici a Git: “*ecco, salva com’è adesso*”. Git scatta una **fotografia** e la mette in fila con le altre. Da quel momento puoi **tornare a qualsiasi fotografia del passato** quando vuoi: com’era ieri, la settimana scorsa, un mese fa.

La cosa bella è che tutte queste fotografie stanno dentro **una sola cartella**, ordinata e pulita. Niente più venti file con nomi strani: uno solo, con dentro **tutta la storia**.

In una frase: Git è un **sistema di controllo di versione** — cioè un programma che tiene la storia di un progetto e ti lascia viaggiare avanti e indietro nel tempo.

Esempio. Stai programmando un videogioco. Funziona tutto, poi provi a cambiare una cosa e… si rompe. Con Git non è un dramma: torni all’ultima fotografia che funzionava e riparti da lì. Zero panico.

3. Un po’ di storia: chi l’ha inventato e perché

Git non è nato in un ufficio di marketing: è nato da un **bisogno reale**.

Nel **2005** un programmatore famoso, **Linus Torvalds** (lo stesso che aveva creato **Linux**, il sistema operativo), aveva un problema grosso. Al codice di Linux lavoravano **migliaia di persone** sparse per il mondo, tutte insieme. Serviva uno strumento per mettere ordine in quel caos. Quello che usavano prima, a un certo punto, non si poté più usare gratis.

Invece di arrendersi, Torvalds fece una cosa da film: **si scrisse lo strumento nuovo da solo, in pochi giorni**. Doveva essere **velocissimo**, non perdere **mai** i dati, e reggere **tantissime persone** che lavorano in parallelo. Quello strumento è Git.

Curiosità. In inglese “git” è una parola scherzosa per dire “testa dura”. Torvalds disse ridendo di dare sempre ai suoi progetti “un nome ispirato a sé stesso”. Oggi quel piccolo programma è lo **standard usato in tutto il mondo del lavoro**.

4. Git e GitHub non sono la stessa cosa

Qui all’inizio si confondono tutti, quindi fermiamoci un attimo. **Git** e **GitHub** sembrano gemelli ma fanno due mestieri diversi.

1. **Git** è lo **strumento** (la macchina del tempo di cui parlavamo). Non è di nessuno: è **gratis e "open source"**, cioè il suo codice è pubblico e chiunque può usarlo.
2. **GitHub** è un **sito web** dove metti i tuoi progetti Git **online**, per averli al sicuro, mostrarli e lavorarci in gruppo. Dietro GitHub c'è un'**azienda vera** (GitHub, Inc.), oggi di proprietà di **Microsoft**.

Analogia della macchina: Git è il **motore** — la tecnologia che fa funzionare tutto. GitHub è il **grande garage online** dove parcheggi le tue auto, le fai vedere agli altri e ci lavorate insieme. Il motore puoi usarlo anche senza quel garage; ma il garage è comodo.

E attenzione: GitHub **non è l'unico garage**. Ci sono anche **GitLab** e **Bitbucket**, che fanno cose simili. Noi useremo GitHub perché è il più diffuso — quando cerchi lavoro, quello che quasi tutti conoscono è lui.

5. GitHub è un "servizio": PaaS e Freemium

Abbiamo detto che GitHub è un servizio online. Due parole che hai visto alla lavagna spiegano **che tipo** di servizio è.

PaaS — "la piattaforma come servizio"

PaaS è la sigla di **Platform as a Service**, cioè "piattaforma **come servizio**". Vuol dire: qualcuno ti mette a disposizione online una **piattaforma già pronta**, e tu la usi senza doverti costruire e mantenere i computer, i server e tutto il resto.

Analogia della cucina. Vuoi aprire un ristorante. Puoi **costruirti la cucina da zero** (comprare forni, frigoriferi, sistemare i tubi… tanta fatica), oppure **affittare una cucina già attrezzata** e pensare solo a cucinare. GitHub è la cucina già pronta: tu porti il tuo progetto, il resto lo mette lui.

Freemium — "gratis + premium"

Freemium è una parola inventata unendo **FREE** (gratis) e **PREMIUM** (a pagamento). Significa: le funzioni **di base sono gratis** per tutti, e se vuoi funzioni **in più** paghi un abbonamento.

Esempio che conosci. Tanti videogiochi per telefono sono così: giochi gratis, ma se vuoi lo scudo dorato o niente pubblicità, paghi. Anche Spotify: ascolti gratis con la pubblicità, oppure paghi il "premium" e la togli. **GitHub funziona allo stesso modo:** a te studente basta e avanza la parte gratis.

La famiglia “come servizio” (…aaS)

PaaS non è da solo: fa parte di una **famiglia** di servizi online, tutti con la sigla che finisce in **“aaS”** (*as a Service*, “come servizio”). Cambia **quanto lavoro fai tu** e quanto lo fa il fornitore. I tre principali, dal “fai più tu” al “fai meno tu”:

Sigla	Nome	Cosa ti danno già pronto	Esempio
IaaS	Infrastructure as a Service	Solo i **muri e la corrente**: computer/server nudi. Il resto lo monti tu.	Amazon AWS, Google Cloud (server in affitto)
PaaS	Platform as a Service	La **cucina già attrezzata**: piattaforma pronta, tu porti solo il progetto.	GitHub, Netlify
SaaS	Software as a Service	Il **piatto già cucinato e servito**: un programma pronto, lo usi dal browser.	Gmail, Google Documenti, Spotify

Come ricordarli — la pizzeria. IaaS = affitti il **locale vuoto** e ti arrangi. **PaaS** = affitti la **cucina attrezzata** e pensi solo a cucinare la pizza. **SaaS** = ti fai **consegnare la pizza pronta** a casa. Più scendi, meno fatica fai (ma decidi meno cose).

Riassunto della lavagna: GitHub è un **servizio** — nello specifico una **piattaforma** (**PaaS**, uno della famiglia …aaS) — dove tieni **repository di codice**, e si paga col modello **Freemium**: gratis per l’uso normale, a pagamento per le cose avanzate.

6. Le parole di Git spiegate facili

Git ha un suo piccolo vocabolario. Sono solo quattro parole importanti: impariamole con un esempio ciascuna.

Repository (repo)

È la **cartella-progetto** che Git tiene d’occhio, con dentro i file **e tutta la loro storia**. Creare un repository vuol dire dire a Git: “*da qui in poi, di questa cartella, tieni la storia*”.

Esempio. Il tuo videogioco “SpaceRun” è un repository: dentro c’è il gioco **e** il diario di tutte le modifiche fatte.

Commit

Un **commit** è una delle nostre **fotografie**: un salvataggio, con attaccato un **messaggio** che dice cosa hai cambiato.

“menu”“punteggio”“grafica”“suoni”

Una fila di commit: ognuno è una foto con un messaggio. Puoi tornare a qualunque punto.

Consiglio: scrivi messaggi chiari (“aggiunto il menu iniziale”). Riletti dopo un mese, sembrano il diario del progetto.

Branch (ramo)

Un **branch** è una **strada parallela**. Serve per provare un’idea nuova **senza rovinare** la versione che funziona (la strada principale si chiama main).

Esempio. Vuoi provare a mettere i colori dark nel gioco, ma non sei sicuro. Apri un branch “colori-scuri”, provi lì. Se piace lo tieni; se no, lo butti e il gioco resta com’era.

Merge (unione)

Quando l’idea del branch funziona, la **unisci** alla strada principale: questo si chiama **merge**. Da quel momento anche il main ha la novità.

7. Come si lavora davvero: la storia di due compagni

Aisha e Marco fanno insieme il gioco “SpaceRun” su GitHub. Guardiamo cosa fanno, passo passo:

1. **clone** — la prima volta ognuno **scarica** il progetto dal sito sul proprio computer.
2. **lavora** — Aisha disegna i nemici, Marco scrive i movimenti. Ognuno sul suo pezzo.
3. **commit** — ogni tanto scattano una fotografia del proprio lavoro, con un messaggio.
4. **push** — **mandano** le loro fotografie **online**, su GitHub, così l’altro le vede.
5. **pull** — **scaricano** le novità dell’altro, per restare aggiornati.

Il tuo computer GitHub (online) push ⟶ pull

E se Aisha vuole proporre a Marco una modifica importante, senza imporgliela? Apre una **Pull Request**: è come dire “*ehi, ho fatto questa cosa, la guardi? se ti va la*

uniamo”. Marco la legge, commenta, e se è d’accordo si fa il merge. Ordinato e senza litigi.

Importante per noi: faremo **tutto dal browser**, cliccando sui bottoni del sito GitHub. Le parole (commit, push, merge…) restano quelle, ma non si digita niente.

8. Il versioning: i numeri 1.2.3 e come si fa

Quando un progetto arriva a un punto “buono”, gli si dà un **numero di versione**. Lo standard più usato al mondo ha la forma **v1.2.3**, e ogni numero racconta una cosa:

Numero	Nome	Aumenta quando…
1 (il primo)	major	cambi **tutto**, in modo grosso, magari incompatibile col vecchio
2 (il secondo)	minor	aggiungi una **funzione nuova**, ma il resto funziona ancora
3 (il terzo)	patch	correggi solo un **bug**, niente di nuovo

Esempio con il gioco. v1.0.0 è la prima versione giocabile ⟶ aggiungi i suoni e diventa v1.1.0 ⟶ trovi un bug e lo correggi: v1.1.1 ⟶ rifai il gioco da zero con un motore nuovo: v2.0.0.

Come si fa, in pratica

1. Lavori e fai i tuoi **commit** (le fotografie) come sempre.
2. Quando il progetto è **pronto e provato**, scegli il numero: si parte da v1.0.0.
3. Metti una **bandierina** (si chiama **tag**) su quel commit con scritto il numero.
4. Su GitHub pubblichi una **Release**: quella versione diventa **ufficiale** e **non cambia più**.
5. Continui a lavorare e, alla modifica dopo, **aumenti il numero** secondo la regola sopra.

Regola d’oro: un numero già pubblicato **non si riusa mai**. Se cambi anche una sola cosa, prima **aumenti** il numero e poi pubblichi. Così due versioni diverse non si confondono mai.

9. Tanti casi d’uso con i grafi

Vediamo Git “in azione” in situazioni vere. In ogni grafo ogni **pallino** è un commit (una foto) e la **linea** è il tempo, che va da sinistra a destra.

Caso A — Lavoro da solo, tutto dritto

Stai facendo un piccolo sito da solo. Salvi passo dopo passo: la storia è una **linea dritta**, semplice semplice.

iniziohome pagecontattifinito

Caso B — Provo un’idea rischiosa

Vuoi provare un’idea che **forse non funziona**. Apri un **branch** a parte: se l’idea è buona la **unisci** (merge); se è un disastro **butti il ramo** e il main resta intatto. Nessun rischio.

main

idea nuova (branch)

merge (se funziona)

Caso C — Bug urgente mentre sto lavorando

Stai costruendo una funzione lunga, ma salta fuori un **bug grave** nella versione già pubblicata. Apri un ramo **hotfix** (correzione veloce), sistemi il bug, lo unisci subito al main — e poi torni tranquillo alla tua funzione, senza aver perso niente.

main

hotfix (bug)

la mia funzione lunga (branch)

Caso D — Squadra: tanti rami e Pull Request

Nel progetto di gruppo ognuno lavora sul **suo ramo** (chi la grafica, chi i movimenti, chi l’audio). Ciascuno apre una **Pull Request**; il gruppo la controlla e la unisce. Il main cresce con i pezzi di tutti, senza pestarsi i piedi.

main

grafica (PR)

movimenti (PR)

audio (PR)

Caso E — Le versioni pubblicate (Release)

Mentre il progetto avanza, si pubblicano le **Release**. Le bandierine sul main segnano le versioni ufficiali che gli utenti (o i compagni) ricevono. È esattamente il grafo che abbiamo disegnato alla lavagna.

v1.0v1.1v1.2v2.0

1. suoni+ livellitutto nuovo

10. Riepilogo e glossario

Se ti ricordi solo tre cose, ricordati queste:

1. Git è la **macchina del tempo** del tuo progetto: salva la storia e ti fa tornare indietro.
2. **Git** è lo strumento; **GitHub** è il sito-servizio (una piattaforma **PaaS**, gratis in modello **Freemium**) dove lo tieni online e collabori.
3. Le **versioni** si scrivono v1.2.3 e una pubblicata non si tocca più: se cambi, aumenti il numero.

Parola	Vuol dire
Git	lo strumento (gratis, open source) che tiene la storia del progetto
GitHub	il sito-servizio (azienda di Microsoft) che ospita i progetti online
PaaS	“piattaforma come servizio”: una piattaforma già pronta che usi online
Freemium	gratis nelle funzioni base, a pagamento per quelle avanzate
repository	la cartella-progetto con tutta la storia
commit	un salvataggio (una fotografia) con un messaggio
branch	una strada di lavoro parallela (ramo)
merge	unire un ramo alla strada principale
clone	scaricare la prima volta un progetto sul proprio computer
push / pull	mandare / scaricare le fotografie da e verso GitHub
Pull Request	proposta di unione, da guardare prima di fare il merge
tag / Release	una versione pubblicata e “congelata”
versioning	dare un numero (v1.2.3) alle versioni del progetto

Classe 3 · Informatica · v1.1 · si scrive in Google Documenti e si consegna su Google Classroom · italiano

Obiettivo: dimostrare che hai capito **cosa sono i tipi di file** (formati, tag, compressione, video) con parole tue. Conta la comprensione, non le frasi perfette. Puoi usare la dispensa per ripassare, ma le risposte devono essere **tue**.

Ripasso lampo

1. L'**estensione** (.mp4, .md…) è il "cognome" del file: dice il tipo e con che programma si apre.
2. I file **a tag** (HTML, XML, MD) marcano il testo con etichette per dargli struttura.
3. **Refresh rate** = quante volte al secondo si aggiorna lo **schermo** (Hz); **fps** = fotogrammi al secondo di un **video**.
4. **Compressione**: senza perdita (ZIP, PNG = identico) o con perdita (JPG, MP4 = più leggero).

Apri un documento nuovo: nel browser vai su docs.new e dai un **titolo** (es. "I tipi di file — Nome Cognome"). Rispondi alle 5 parti.

Parte 1 — Le basi (con parole tue)

1. Cos'è un **file** e a cosa serve l'**estensione**? (2-3 frasi)
2. Perché rinominare foto.jpg in foto.txt non la trasforma in testo?

Parte 2 — Tre formati diversi

1. Scegli **tre formati** di famiglie diverse (es. uno di testo, uno immagine/video, uno archivio) e per ognuno scrivi: **a cosa serve** e **con che programma si apre**.

Parte 3 — Un file a tag

1. Scrivi un **piccolo esempio** (3-4 righe) di un file a tag a tua scelta (**HTML** o **XML** o **MD**) e spiega cosa fanno i tag che hai usato. Esempio di partenza da modificare:

<h1>Il mio titolo</h1> Un paragrafo con una parola in grassetto.

Parte 4 — Compressione

1. Spiega la differenza tra compressione **senza perdita** e **con perdita**, con **un esempio** per ciascuna (quale formato e in quale situazione la useresti).

Parte 5 — Schermo e video

1. Con parole tue: che differenza c’è tra il **refresh rate** di un monitor (Hz) e gli **fps** di un video? Perché sono due cose diverse ma collegate?

Parte 6 — Testo o binario, e il percorso

1. Come fai a capire se un file è di **testo** o **binario**? Fai **un esempio** di ciascuno.
2. Scrivi il **percorso (path)** di un file d’esempio sul computer (es. C:\Documenti\nomefile.txt).

Quando hai finito, **consegna** il documento su **Classroom**.

Nota: conta soprattutto **saperlo spiegare a voce**, con parole tue.

Changelog

1. **v1.0 (29/10/2026)**: prima versione, dalle fonti della lezione.